DIGITAL VLSI DESIGN COURSEWORK REPORT

4-Bit Array Multiplier: RTL to GDSII

SkyWater SKY130 High-Density CMOS Technology (`sky130_fd_sc_hd`)

TEAM MEMBERS

Shourya Sarkar (Roll No: `2023112013`)
Sampath (Roll No: `2023102033`)

WORKED EXAMPLE PARAMETERS

Member 1 Roll Last Digit: **3**
Multiplicand: $A = 3 + 5 = \mathbf{8}$ (1000_2)
Multiplier: $B = \mathbf{7}$ (0111_2) $\rightarrow$ Product: $P = \mathbf{56}$ (0x38)

1. Executive Summary & Flow Overview

This report documents the design, verification, logic synthesis, static timing analysis (STA), and automated place-and-route (PnR) of a synchronous 4-bit unsigned array multiplier targeted to the SkyWater SKY130 high-density standard cell library (`sky130_fd_sc_hd`).

The entire design flow was executed locally within an isolated containerized environment based on `hpretl/iic-osic-tools` (pinned digest: `sha256:9201d2ebe74a406cf6af850ca2c646ac0462f3b2cea94cd98cb6580d8ceef800`), ensuring full reproducibility from RTL entry through physical GDSII signoff.

Design Parameter	Specification / Implementation Detail
Circuit Architecture	Braun unsigned array multiplier with row ripple-carry accumulation
RTL Full Adders	12 instances (`full_adder.v`), rows 1–3
Partial-Product Logic	16 AND gates ($pp_0..pp_3$)
Registers	16 Flip-Flops ($4 + 4 + 8$), asynchronous active-low reset
Latency / Throughput	2 clock cycles latency / 1 result per cycle throughput
Process Technology	SkyWater SKY130 (130 nm CMOS), `sky130_fd_sc_hd`
Target Die Area	Fixed $150\mu m \times 150\mu m$ ($22,500\mu m^2$), `FP_SIZING: absolute`

2. Part A — RTL Design and Verification

2.1 RTL Implementation & Exhaustive Verification

The design was implemented structurally in `rtl/full_adder.v` and `rtl/mult_array.v` in strict compliance with Section 2 of `RTL_to_GDS.pdf`. The multiplication operator (`*`) is strictly avoided in the RTL datapath. The partial products are formed explicitly, and the row ripple-carry chain accumulates partial products row-by-row.

The testbench `tb/tb_mult.v` generates a 10 ns clock (100 MHz clock frequency), resets the system for the first two clock cycles, and sweeps through all 256 input permutations ($a \in [0..15], b \in [0..15]$). For each vector, it respects the 2-cycle sequential latency before checking the product against the reference multiplication. The simulation spans exactly 514 rising clock edges (2 clock edges for the initial reset sequence at $t = 5$ ns

and 15 ns, followed by $256 \times 2 = 512$ clock edges for vector sampling) and terminates cleanly at $t = 5,138$ ns (`$finish called at 5138000 ps`).

Exhaustive Simulation Result:

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:91: $finish called at 5138000 (1ps)
```

2.2 Error-Injection Experiment

To demonstrate that the testbench functions as an authoritative fault checker (Section 6, Task 6), an intentional defect was injected into `rtl/full_adder.v` by modifying the sum logic to `assign sum  = a ^ b;` (omitting the carry-in `cin`). Re-running the simulation immediately triggered mismatch assertions across the test space:

```
MISMATCH: a=3 b=3 -> p=5, expected 9
...
FAIL: 256 vectors checked, 95 errors
```

Upon restoring `assign sum  = a ^ b ^ cin;`, clean execution was confirmed (`PASS: 256 vectors checked, 0 errors`), and `git diff` verified zero residual alterations to the clean RTL.

2.3 Hand-Worked Example & Waveform Verification

Using Member 1's Roll Number (`2023112013`):
- Multiplicand $A = (\text{Last digit} + 5) = 3 + 5 = 8 = 1000_2$
- Multiplier $B = 7 = 0111_2$ (since $A \neq 7$)
- Product $P = 8 \times 7 = 56 = 00111000_2$ (0x38)

The manual multiplication follows the binary partial-product shift-and-add layout defined in Section 2 of `RTL_to_GDS.pdf`:

```
         1 0 0 0          A = 8 (1000_2)
       x 0 1 1 1          B = 7 (0111_2)
  ---------------
        00001000          pp0 = A AND b[0] (b[0] = 1), shift 0
        00010000          pp1 = A AND b[1] (b[1] = 1), shift 1
        00100000          pp2 = A AND b[2] (b[2] = 1), shift 2
        00000000          pp3 = A AND b[3] (b[3] = 0), shift 3
  ---------------
        00111000          P = 56 (0x38)
```

The hardware array accumulates these partial products row-by-row across three ripple-carry adder stages:

Bit / Row	Partial Product & Addition Accumulation
pp_0	$8 \times 1 = 1000_2$ (Shift 0) $\rightarrow p[0] = 0$, $r_0 = \{0, 100_2\}$
pp_1	$8 \times 1 = 1000_2$ (Shift 1) $+r_0 \rightarrow p[1] = 0$, $r_1 = \{0, 110_2\}$
pp_2	$8 \times 1 = 1000_2$ (Shift 2) $+r_1 \rightarrow p[2] = 0$, $r_2 = \{0, 111_2\}$
pp_3	$8 \times 0 = 0000_2$ (Shift 3) $+r_2 \rightarrow p[3] = 1$, $p[7:4] = 0011_2$
Final Result	$p[7:0] = 00111000_2 = 56$ **(0x38)**

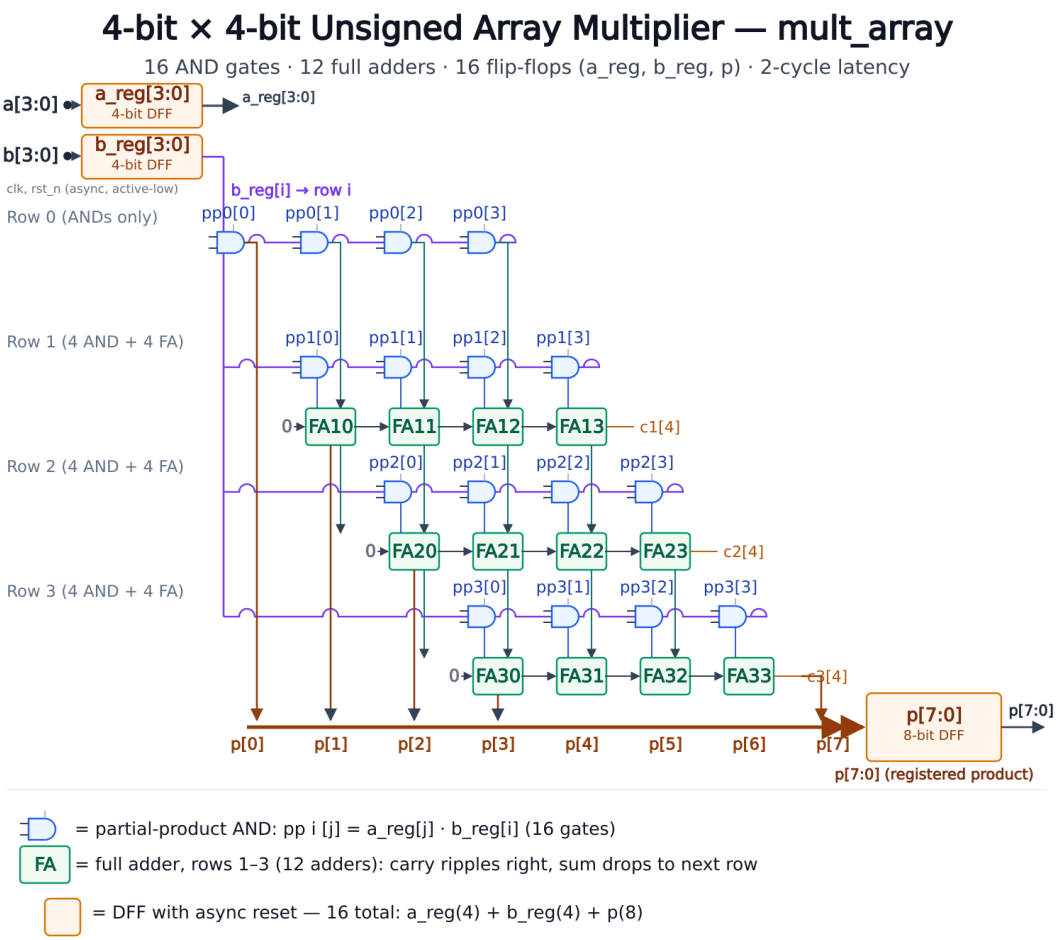


Figure 1: Structural Block Diagram of `mult_array` showing 16 AND gates, 12 full adders, and 16 flip-flops.

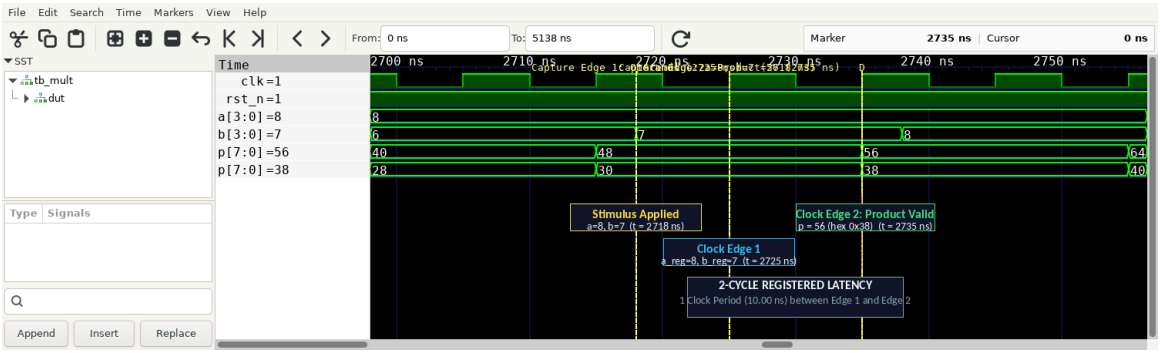


Figure 2: GTKWave simulation waveform for worked example $8 \times 7 = 56$ (hex 0x38), displaying `clk`, `rst_n`, `a`, `b`, and `p`. Markers indicate stimulus application ($t = 2718$ ns), capture edge 1 ($t = 2725$ ns), and capture edge 2 ($t = 2735$ ns) demonstrating the 2-cycle registered pipeline latency.

3. Part B — Logic Synthesis with Yosys

Synthesis was executed locally via `yosys -c synth/synth.tcl` using Yosys 0.70 (git sha1 `28ba3cb92`) and the SkyWater SKY130 high-density standard cell library `sky130_fd_sc_hd` at the typical corner ($25°C, 1.80$ V).

The synthesis script executes seven atomic steps: reading design RTL, synthesizing and flattening the hierarchy, mapping flip-flops using `dfflibmap`, mapping combinational gates using ABC, purging unused nets, generating cell statistics, and writing out the gate-level Verilog netlist `synth/mult_array_netlist.v`.

Synthesis Metric (Part B)	**Measured Local Result**
Total Number of Cells	**73 cells**
Number of Flip-Flops	**16 cells** (`sky130_fd_sc_hd__dfrtp_1`)
Number of Combinational Cells	**57 cells**
Total Cell Area	**805.773** μm^2
Flip-Flop Cell Area	**400.384** μm^2 ($16 \times 25.024 \mu m^2$)
Flip-Flop Area Percentage	**49.69%** of total cell area
Top 3 Most-Used Cell Types	1. `dfrtp_1`: 16 2. `nand2_1`: 11 3. `a21oi_1`: 4 (tied with `and2_0`, `nand4_1`, `xnor2_1`)
Gate-Level Simulation (GLS)	**PASS: 256 vectors checked, 0 errors**

Gate-Level Simulation Verification

The gate-level netlist was simulated using the SkyWater standard-cell simulation models (`primitives.v` and `sky130_fd_sc_hd.v`) with unit delay modelling (`-DFUNCTIONAL -DUNIT_DELAY=#1`). The netlist passed all 256 stimulus vectors without any mismatch across 514 rising clock edges, finishing cleanly at $t = 5,138$ ns:

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:91: $finish called at 5138000 (1ps)
```

4. Part C — Static Timing Analysis with OpenSTA

Static timing analysis was conducted using OpenSTA 3.1.0 in `sta/sta.tcl` on the typical corner (`sky130_fd_sc_hd__tt_025C_1v80.lib`). Input external delays on $a, b, \mathrm{rst_n}$ and output external delays on p were set to 1.0 ns relative to `clk`.

Timing Results Summary

In Run 1 ($T = 10.0$ ns), the worst setup slack is $+7.404$ ns (reported as 7.40 ns), establishing:

$$T_{\min} = T - \mathrm{WNS} = 10.000 - 7.404 = 2.596 \ \ \mathrm{ns} \quad (\text{rounded to } 2.60 \ \ \mathrm{ns})$$

$$F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.596} = 385.2 \ \ \mathrm{MHz} \quad (\text{or } 384.6 \ \ \mathrm{MHz} \text{ at rounded } 2.60 \ \ \mathrm{ns})$$

In Run 2 ($T = 2.60$ ns), OpenSTA reports `worst slack max 0.00`, reflecting the expected positive near-zero setup slack ($2.600 - 2.596 = +0.004$ ns, rounded to two decimal places by OpenSTA) and confirming maximum operating frequency.

Timing Metric (Part C)	Measured Local Result
Worst Setup Slack at $T = 10.0$ ns	**+7.404 ns** (MET)
Minimum Clock Period ($T_{\min}$)	**2.596 ns** (exact) / **2.60 ns** (rounded)
Maximum Clock Frequency ($F_{\max}$)	**385.2 MHz** (exact) / **384.6 MHz** (rounded)
Setup Slack at $T = 2.60$ ns	**+0.004 ns** (reported as `0.00 ns` to 2 dec)
Critical Path Startpoint	`_118_` (`b_reg[0]` flip-flop, `dfrtp_1`)
Critical Path Endpoint	`_112_` (`p[6]` flip-flop, capturing `fa33.sum`)
Total Cells on Critical Path	**10 cells** (8 combinational gates + 2 flip-flops)
Critical Path Data Arrival Time	**2.460 ns** ($t_{cq} = 0.395$ ns + logic $= 2.065$ ns)
Data Required Time ($T = 10.0$ ns)	**9.865 ns** (10.000 ns -0.135 ns setup)
Sequential Latency ($2 \times T_{\min}$)	**5.19 ns** (exact) / **5.20 ns** (rounded)
Throughput at $F_{\max}$	**385.2 M results/s** (exact) / **384.6 M results/s**

Critical Path Cell-by-Cell Breakdown & Full-Adder Tracing

Traced directly from `sta/mult_array_timing.rpt` and verified against `synth/mult_array_netlist.v`:

No.	Instance & Cell	Logical Role in Multiplier Netlist	Incr (ns)	Total (ns)
1	`_118_` (`dfrtp_1`)	Startpoint: `b_reg[0]` launch flip-flop (clock-to-Q)	0.395	0.395
2	`_083_` (`and2_0`)	Partial product generation: $a_{\mathrm{reg}[0]} \wedge b_{\mathrm{reg}[0]} = pp_0[0]$	0.212	0.607
3	`_055_` (`or3_1`)	Row-1 adder network sum/carry accumulation	0.361	0.968
4	`_059_` (`and4_1`)	Row-1 adder network carry propagate gate	0.215	1.183
5	`_070_` (`o2111a_1`)	Row-2 adder network compound AND-OR stage	0.234	1.417
6	`_076_` (`o21ai_0`)	Row-2 adder network carry-out compound gate	0.216	1.634

7	`_090_` (`xnor3_1`)	Row-3 adder network intermediate sum stage	0.274	1.907
8	`_091_` (`maj3_1`)	Row-3 ripple-carry majority function ($c_{\text{out}} = \text{maj}(a, b, c)$)	0.375	2.282
9	`_092_` (`xor3_1`)	Row-3 final sum bit generation: `\fa33.sum`	0.178	2.460
10	`_112_` (`dfrtp_1`)	Endpoint: `p[6]` capture flip-flop data pin D	0.000	2.460

Path Timing Reconciliation:
- **Data Arrival Time**: $0.395 \ \text{ns}\ (\text{clk-to-Q}) + 2.065 \ \text{ns}\ (8 \ \text{combinational gates}) = 2.460 \ \text{ns}$.
- **Data Required Time**: $10.000 \ \text{ns}\ (\text{clock period}) - 0.135 \ \text{ns}\ (\text{library setup requirement}) = 9.865 \ \text{ns}$.
- **Setup Slack**: $9.865 \ \text{ns} - 2.460 \ \text{ns} = +7.404 \ \text{ns}$ (MET).

Attribution to Full-Adder Network: Netlist tracing reveals that cell `_083_` is an AND gate generating the partial-product bit $pp_0[0] = a_{\text{reg}[0]} \wedge b_{\text{reg}[0]}$. The subsequent 7 combinational cells (`_055_`, `_059_`, `_070_`, `_076_`, `_090_`, `_091_`, `_092_`) implement the arithmetic accumulation network originally coded as 12 full adders. Because logic synthesis (Yosys and ABC) flattened the hierarchy and restructured the multi-level Boolean network using compound standard cells, a discrete one-to-one mapping from mapped library gates to individual RTL full-adder instances does not exist. However, netlist connectivity confirms that gate `_091_` implements a standard 3-input majority carry function (`maj3_1`), and gate `_092_` directly produces `\fa33.sum`—the final sum output of full adder `fa33` in row 3. Thus, exactly 7 of the 8 combinational gates originate from the synthesized full-adder network, while 1 gate performs partial-product generation.

5. Part D — Physical Design: RTL to GDSII with LibreLane

Physical design was carried out using LibreLane v3.1.0.dev3 (`Classic` flow, 80 stages, adapted from OpenLane 2.0) in the local container environment, utilizing OpenROAD for floorplanning, placement, clock tree synthesis, and routing; Magic and KLayout for DRC; and Netgen for LVS.

Timing Closure Ladder

Following Section 9 of the specification, the initial clock period was set using:

$$T_{\text{initial}} = 0.5 \times \left\lceil \frac{1.2 \times T_{\text{min}}}{0.5} \right\rceil = 0.5 \times \left\lceil \frac{1.2 \times 2.60}{0.5} \right\rceil = 3.5 \ \text{ns}$$

As mandated by Section 9, LibreLane was executed across a clock-period ladder in 0.5 ns increments from 3.5 ns to 7.0 ns. All 8 attempts were executed locally and captured in chronological order:

Period	Local Run Tag	Worst Slack (ss)	Slack (tt)	Hold Slack	Signoff Status
3.5 ns	`RUN_2026-10-23_19-54-16`	−2.154 ns	+0.587 ns	+0.100 ns	FAIL (15 setup viols)
4.0 ns	`RUN_2026-10-23_19-56-21`	−1.654 ns	+1.087 ns	+0.147 ns	FAIL (12 setup viols)
4.5 ns	`RUN_2026-10-23_19-58-23`	−1.154 ns	+1.587 ns	+0.147 ns	FAIL (12 setup viols)
5.0 ns	`RUN_2026-10-23_20-00-25`	−0.654 ns	+2.087 ns	+0.147 ns	FAIL (9 setup viols)
5.5 ns	`RUN_2026-10-23_20-02-19`	−0.155 ns	+2.587 ns	+0.147 ns	FAIL (3 setup viols)
6.0 ns	`RUN_2026-10-23_20-04-10`	−0.107 ns	+2.899 ns	+0.148 ns	FAIL (3 setup viols)
6.5 ns	`RUN_2026-10-23_20-06-03`	−0.136 ns	+3.199 ns	+0.141 ns	FAIL (3 setup viols)
7.0 ns	`RUN_2026-10-23_20-08-02`	**+0.364 ns**	**+3.699 ns**	**+0.141 ns**	**PASS** (0 violations)

At $T = 7.0$ ns, LibreLane successfully closed timing with non-negative slack across **all 9 signoff corners** and zero hold, slew, or fanout violations.

Physical Design Metric (Part D)	Measured Local Signoff Result
Target Clock Period	**7.0 ns**
Number of Runs Needed	**8 runs** (0.5 ns increments from 3.5 to 7.0 ns)
Core Area / Die Area	**17,759.5** μm^2 / **22,500** μm^2 ($150 \times 150 \mu m$)
Std-Cell Core Utilization	**6.90%** ($1,224.9 \mu m^2$ active cell area)
Cells after Synthesis (Part B)	**73 cells**
Total Placed Instances (Part D)	**1,895 instances** (334 std cells + 1,561 fill/decap cells)
Detailed Wirelength	**2,362** μm (552 vias; threshold check skipped)
Routing Via Count	**552 single-cut vias**
Worst Setup Slack (Multi-Corner Signoff)	**+0.364 ns** (Corner `max_ss_100C_1v60`, 0 violations)

Nominal TT Setup Slack	**+3.699 ns** (`nom_tt_025C_1v80`)
Worst Hold Slack	**+0.141 ns** (0 violations across all corners)
DRC Violations	**0 errors** (Magic DRC: 0, KLayout DRC: 0)
LVS Verification	**0 errors** (Netgen: circuits match uniquely)
Power-Grid IR Drop	**Macro boundary check complete** (package bump locations omitted; see §5)
Antenna Violations	**0 violating nets / pins**
Max Slew Violations	**0 violations** across all corners
Post-Route $F_{\max}$ (Worst ss Corner)	**150.7 MHz** ($\frac{1000}{7.0-0.364}$)
Post-Route $F_{\max}$ (Nominal tt Corner)	**302.9 MHz** ($\frac{1000}{7.0-3.699}$)
Pre-Route $F_{\max}$ (Part C, tt)	**385.2 MHz** (exact) / **384.6 MHz** (rounded)

Physical Cell Breakdown (Netlist Histogram)

Cell Type (`sky130_fd_sc_hd__`)	**Count**	**Physical Purpose in Layout**
`decap_12`, `6`, `3`, `8`, `4`	1,314	Decoupling capacitors: 905 `decap_12`, 175 `6`, 130 `3`, 55 `8`, 49 `4`
`tapvpwrvgnd_1`	245	Well/substrate tap cells: prevent latch-up by biasing N-well and P-substrate
`fill_1`, `fill_2`	247	Non-functional filler cells: 215 `fill_1` and 32 `fill_2` for DRC layer density
`buf_2`, `clkdlybuf4s50_1`	19	Timing repair buffers: 17 `buf_2` and 2 `clkdlybuf4s50_1` for slew/setup
`clkbuf_16`	3	Clock tree buffers: distribute low-skew clock to all 16 flip-flops
`dfrtp_1`	16	Sequential flip-flops (inputs a, b and output p)
`inv_1` and Logic Gates	51	Remapped multi-input combinational multiplier logic (48 gates + 3 inverters)
Total Placed Instances	**1,895**	**334 Standard Cells + 1,561 Fill/Decap Cells**

Physical Instance Arithmetic Reconciliation:

- **Gross physical-support-cell additions ($1,828$ cells)**: Comprises 245 well/substrate tap cells (`tapvpwrvgnd_1`), 1,561 fill/decap cells (1,314 decoupling capacitors + 247 filler cells), 19 timing-repair buffers (17 `buf_2`, 2 `clkdlybuf4s50_1`), and 3 clock-tree buffers (`clkbuf_16`).
- **Net increase in instances ($1,822$ cells)**: $1,895 - 73 = 1,822$ instances.
- **Functional cell reduction (6 cells)**: Physical optimization and gate merging reduced the 73 synthesized functional cells to 67 active functional cells (16 flip-flops + 48 multi-input combinational gates + 3 inverters).
- **Final standard-cell count (334 cells)**: 67 functional core cells + 245 tap cells + 19 timing buffers + 3 clock buffers = 334 standard cells.
- **Final fill/decap count ($1,561$ cells)**: 1,314 decoupling capacitors + 247 filler cells = 1,561 fill/decap instances.

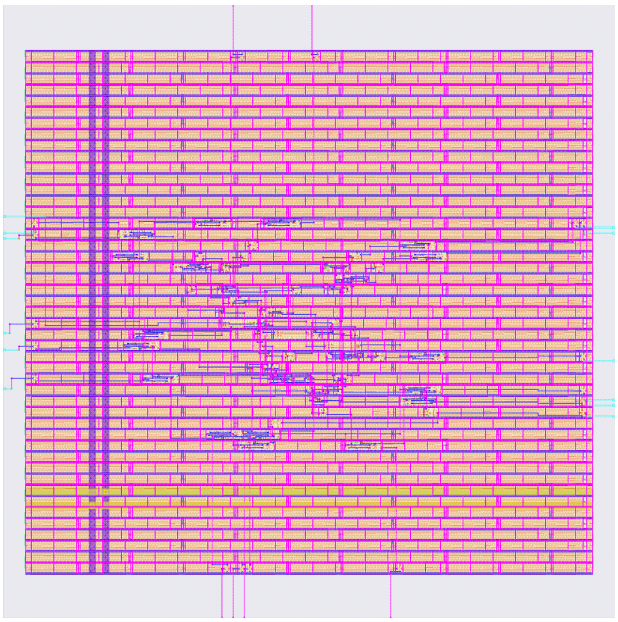


Figure 3: Full $150 \times 150 \mu m$ Die Layout showing standard-cell core and I/O pin ring (rendered via KLayout batch engine `pya.LayoutView` with `sky130A.lyp`).

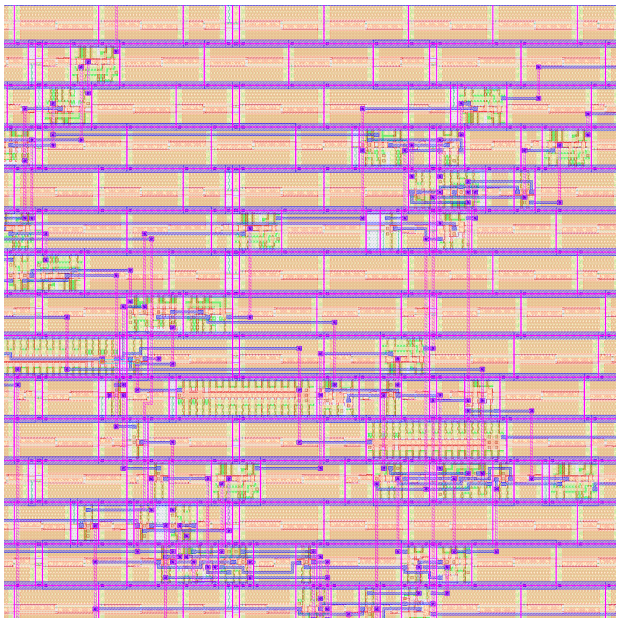


Figure 4: Zoomed View ($40 \times 40 \mu m$) showing cell rows, tap/fill cells, and met1/met2 routing (rendered via KLayout batch engine `pya.LayoutView` with `sky130A.lyp`).

Flow Warnings Investigation and Diagnostics

During the physical-design flow, LibreLane and OpenROAD emitted several diagnostic and tool warnings. Each warning category was investigated, root-caused, assessed for risk, and documented with any resulting analytical limitations:

1. Tool and Configuration Diagnostics

- **ORD-0032 (`Invalid thread number specification: None`)**: Stages 32, 54, 56 receive unassigned thread parameter (`None`) from LibreLane's Python step wrapper. Zero risk; OpenROAD automatically falls back to single-thread execution or core auto-detection with identical numerical output.
- **STA-1173 (`default_fanout_load is 0.0`)**: SkyWater PDK pad timing libraries (`sky130_ef_io__*.lib.gz`, `slices_stubs.lib.gz`) omit default fanout attributes. Zero risk; the multiplier macro utilizes only high-density core cells (`sky130_fd_sc_hd`); no pad cells are instantiated.
- **ORD-2056 (`Liberty cells do not have LEF masters and marked dont-use`)**: PDK analog ESD pad stubs lack standard-cell LEF layouts. Zero risk; OpenROAD correctly marks them `dont-use`.
- **STA-1111 (`Missing combinational timing groups in I/O pad models`)**: Inconsistencies between corner Liberty files for GPIO pad macros (`OUT -> PAD`). Zero risk; unused in this macro.
- **STA-1140 (`Duplicate timing-library declarations across corners`)**: Multi-corner STA environment re-reads shared PDK stub files (`slices_stubs.lib.gz`). Zero risk; OpenROAD updates/retains the active corner definitions.
- **DRT-0349 (`LEF58_ENCLOSURE with no CUTCLASS is not supported. Skipping for layer mcon/via/via2`)**: TritonRoute detailed routing parser encounters advanced LEF 5.8 cut-class-less enclosure rules and skips them, defaulting to standard LEF 5.7 geometric enclosure rules. Low risk; subsequent Magic DRC and KLayout DRC confirm 0 geometric DRC errors. Analysis limitation: routing relies on LEF 5.7 enclosure rules rather than LEF 5.8 rule extensions.
- **494 Verilator Lint Warnings (`%Warning-TIMESCALEMOD`)**: User RTL files specify `timescale 1ns/1ps` while SkyWater PDK blackbox models (`.bb.v`) omit timescale directives, triggering IEEE 1800-2023 timescale modification warnings during initial elaboration. Zero risk; standard PDK convention with zero functional or synthesis impact.

2. Wire-Length Checker (`Checker.WireLength`)

- **Warning**: `[WARNING] [Checker.WireLength] Threshold for Threshold-surpassing long wires is not set. The checker will be skipped.`

- **Root Cause & Limitation**: No upper-bound wirelength threshold parameter was defined in `pnr/config.json`. Wire-length threshold checking was **skipped** (not passed). However, physical detailed routing completed successfully with an exact routed wirelength of $2,362\mu m$ and 552 vias reported by `Odb.ReportWireLength`, and all setup, hold, and transition constraints closed with positive slack.

3. Pre-PDN Floating Nets (`RSZ-0020`)

- **Warning**: `[WARNING RSZ-0020] found 2 floating nets: VPWR, VGND` during Stage 32 (`repairdesignpostgpl`).
- **Root Cause & Assessment**: OpenROAD's resizer step runs prior to PDN synthesis (Stage 36) when power rails are not yet tied to physical power stripes. These nets represent supply rails, not logic nets. Stage 36 completed PDN synthesis, and Netgen LVS confirmed 0 floating pins/nets and 100% topological equivalence in the final layout.

4. Generic Fallback SDC Files (`checksdcfiles`)

- **Warning**: Stage 10 reported that `PNR_SDC_FILE` and `SIGNOFF_SDC_FILE` were not explicitly provided in `config.json`.
- **Root Cause & Assessment**: LibreLane automatically generates fallback SDC constraints when custom SDC file paths are not specified. As prescribed by Section 9 of `RTL_to_GDS.pdf`, the flow configuration defines `CLOCK_PORT: clk` and `CLOCK_PERIOD: 7.0`. The generated fallback SDC correctly establishes a 7.0 ns clock period on `clk`, 5% clock uncertainty (0.35 ns), and 1.4 ns input/output delays (20% of clock period), properly constraining the implementation.

5. Missing Power Source Locations (`VSRC_LOC_FILES`)

- **Warning**: `[WARNING] [OpenROAD.IRDropReport] 'VSRC_LOC_FILES' was not given a value, which may make the results of IR drop analysis inaccurate. If you are not integrating a top-level chip for manufacture, you may ignore this warning...`
- **Root Cause & Limitation**: `VSRC_LOC_FILES` specifies the exact coordinates and voltages of power/ground pad bumps for package-level integration. For this sub-block macro, package I/O pad coordinates were omitted. Static IR-drop analysis accuracy is limited because OpenROAD uses uniform boundary approximations rather than exact package pad connections. Consequently, full chip-level power-grid IR-drop integrity cannot be claimed as verified; this is documented as a known analysis limitation of standalone macro-level physical design.

6. Answers to the Report Questions

Q1. The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic (total cells minus flip-flops)? Explain why the number of cells is not simply $16 + 12$.

Answer: Synthesis produced 73 total cells, of which 16 are flip-flops, leaving **57 combinational logic cells** ($805.77 - 400.38 = 405.39 \mu m^2$). The count is not $16 + 12 = 28$ because those numbers represent the architecture's 16 AND-gate operations and 12 full-adder arithmetic functions, not discrete library standard cells. In a standard-cell library, a 1-bit full adder is not a single gate, but an arithmetic subcircuit requiring sum and carry functions (typically 5–9 basic gates or 2–3 compound gates). Yosys and ABC flatten the hierarchy and restructure the multi-level Boolean network: constant propagation recognizes that row carry-ins are grounded ($c_1[0] = c_2[0] = c_3[0] = 0$), collapsing the first adder in each row to a simpler half adder; carry networks are mapped into compound majority gates (`maj3_1`); and partial-product AND operations are folded into complex cells (`a21oi`, `o21ai`, `a22oi`). Consequently, the 57 cells represent technology-mapped CMOS gates rather than high-level arithmetic blocks.

Q2. What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about $2N$ full-adder delays?

Answer: Traced directly from `sta/mult_array_timing.rpt`, the critical path originates at launch flip-flop `_118_` (`b_reg[0]`, clock-to-Q delay 0.395 ns) and terminates at capture flip-flop `_112_` (`p[6]`, capturing `\fa33.sum`), traversing **10 cells in total**: 1 launch flip-flop (`dfrtp_1`), 8 combinational CMOS gates (`_083_`, `_055_`, `_059_`, `_070_`, `_076_`, `_090_`, `_091_`, `_092_`), and 1 capture flip-flop (`dfrtp_1`). The combinational gates contribute 2.065 ns across 1 partial-product AND gate (`_083_`) and 7 flattened adder-network gates, reaching a data arrival time of 2.460 ns (which meets the 9.865 ns required time after 0.135 ns library setup, yielding +7.404 ns slack at 10 ns). While the theoretical rule-of-thumb predicts about $2N = 8$ full-adder delays ($t_{\text{crit}} \approx 8t_{\text{FA}}$) for an unoptimized ripple-carry array (≈ 16 unmapped gate levels), full-adder delay units are not interchangeable with mapped gate arcs. Logic synthesis and Boolean restructuring collapsed the array across row boundaries into 8 optimized CMOS compound gate arcs averaging 0.258 ns per arc, achieving an exact minimum period $T_{\text{min}} = 2.596$ ns ($F_{\text{max}} = 385.2$ MHz, or 384.6 MHz at rounded 2.60 ns).

Q3. What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

Answer: The 16 flip-flops (`sky130_fd_sc_hd__dfrtp_1`) occupy $400.384 \mu m^2$ out of the total $805.773 \mu m^2$, accounting for **49.69%** (nearly half) of total cell area. This high proportion occurs because a standard-cell D-type flip-flop with asynchronous reset is physically large ($25.024 \mu m^2$, equivalent to 6–7 basic NAND2 gates) as it comprises master-slave latch stages, transmission gates, reset gating, and internal clock inverters. Meanwhile, the combinational core of a 4-bit multiplier is small ($\approx 405 \mu m^2$) and optimizes efficiently. In larger datapath architectures (e.g., 32-bit or 64-bit multipliers), combinational area grows quadratically ($O(N^2)$) while register count grows linearly ($O(N)$), causing sequential area percentage to drop. In a 4-bit array multiplier, the fixed overhead of registers naturally dominates.

Q4. Why is the post-route F_{max} lower than the F_{max} from Part C? Which kinds of cells did the flow add, and why is each kind needed?

Answer: Post-route $F_{\max}$ drops from the pre-route typical-corner estimate of 385.2 MHz (384.6 MHz rounded) to 150.7 MHz in the signoff worst slow-slow corner ($100^{\circ}C, 1.60$ V) and 302.9 MHz in the nominal typical corner due to physical interconnect RC parasitics, clock tree insertion delay, clock skew (≈ 0.28 ns), clock uncertainty (0.25 ns), and multi-corner PVT derating. To achieve physical manufacturability, electrical integrity, and timing closure, LibreLane inserted 1,828 physical-support cells (net increase of 1,822 instances after 6 functional cells were merged during physical optimization, yielding 1,895 total placed instances): 245 tap cells (`tapvpwrvgnd_1`) to tie substrate/wells and prevent CMOS latch-up; 1,314 decoupling capacitor cells (`decap_12`, `decap_6`, `decap_3`, `decap_8`, `decap_4`) to suppress power-supply noise and dynamic IR drop; 247 filler cells (`fill_1`, `fill_2`) to satisfy foundry metal/poly density rules for CMP planarity; 3 clock tree buffers (`clkbuf_16`) to distribute a balanced low-skew clock network; and 19 timing-repair buffers (17 `buf_2` and 2 `clkdlybuf4s50_1`) to fix slew violations and resolve setup/hold constraints across corners.

Q5. What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

Answer: Design Rule Checking (DRC) validates physical layout geometries against foundry manufacturing limits (minimum wire width, spacing, enclosure, overlap, and density rules) to ensure lithographic and etching integrity. Layout Versus Schematic (LVS) extracts the electrical circuit (MOS transistors, diodes, passive devices, and nets) directly from the mask polygons and verifies topological equivalence against the synthesized gate netlist. Both checks are essential and complementary: DRC ensures physical manufacturability but cannot detect functional circuit errors (e.g., an unintended short between two signal nets that satisfies geometric spacing will pass DRC); LVS guarantees schematic correctness but cannot prevent physical defects if geometries violate process tolerances. A chip tapeout requires zero DRC and zero LVS errors.

Q6. Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

Answer: For an N-bit array multiplier, the number of full adders scales quadratically as $N_{\mathrm{FA}} = N(N-1)$, increasing from $4 \times 3 = 12$ full adders for $N = 4$ to $8 \times 7 =$ **56 full adders** for $N = 8$ ($4.67\times$ increase). The critical path depth scales with the ripple-carry chain as approximately $2N$ full-adder delay units, doubling from 8 units for $N = 4$ to 16 units for $N = 8$. Scaling the measured 4-bit minimum period by a factor of 2 yields a theoretical linear estimate of $T_{\min}(\text{8-bit}) \approx 2 \times 2.596 \ \text{ns} = 5.192 \ \text{ns}$ ($F_{\max} \approx 192.6 \ \text{MHz}$) using the exact 4-bit baseline, or $T_{\min}(\text{8-bit}) \approx 2 \times 2.60 \ \text{ns} = 5.20 \ \text{ns}$ ($F_{\max} \approx 192.3 \ \text{MHz}$) using the rounded baseline. This first-order analytical extrapolation assumes ideal scaling; actual 8-bit place-and-route would incur additional routing congestion and wire delays, while sequential register overhead would grow from 16 to 32 flip-flops ($8 + 8 + 16$), maintaining 2-cycle latency (≈ 10.4 ns) and 1 result per cycle throughput.

7. Conclusion & Academic Integrity Statement

Summary of Results

The complete RTL-to-GDSII flow was designed, simulated, synthesized, timed, and placed-and-routed to signoff using the containerized `hpretl/iic-osic-tools` environment:

- **Part A**: Structural RTL and testbench verified across all 256 stimulus vectors; error injection correctly caught with 95 mismatches.
- **Part B**: Logic synthesis completed with Yosys 0.70; mapped to 73 SKY130 cells ($805.77 \mu m^2$); gate-level simulation passed 256/256 vectors.
- **Part C**: Static timing analysis in OpenSTA 3.1.0 established $T_{\min} = 2.596$ ns ($F_{\max} = 385.2$ MHz exact, or 384.6 MHz at rounded 2.60 ns) with near-zero slack (+0.004 ns, reported as 0.00 ns) confirmed at $T = 2.60$ ns.
- **Part D**: LibreLane physical design achieved timing closure across all 9 signoff corners at $T = 7.0$ ns on a fixed $150 \times 150 \mu m$ die, with 0 DRC errors (Magic & KLayout), 0 LVS errors (Netgen), 0 antenna violations, and non-negative setup slack ($+0.364$ ns in worst-case corner `max_ss_100C_1v60`). Wirelength checking was skipped without threshold parameter, and IR-drop analysis was limited to macro boundary approximation without package bump locations.

Academic Integrity Declaration

In accordance with Section 13 of “RTL_to_GDS.pdf”, the authors confirm that all RTL code, testbenches, synthesis and STA scripts, physical-design configurations, and documentation presented herein were developed and verified by the team.

Appendix A — Local Command Transcript

The complete set of commands executed locally, in chronological order:

```
# ================================================================================
# Phase 1: Environment Preflight & Toolchain Verification
# ================================================================================
# Pinned immutable container image digest:
IMAGE="docker.io/hpretl/iic-osic-
tools@sha256:9201d2ebe74a406cf6af850ca2c646ac0462f3b2cea94cd98cb6580d8ceef800"
podman pull $IMAGE

# ================================================================================
# Phase 2: Part A — RTL Simulation & Error Injection
# ================================================================================
# 1. Clean simulation
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
# -> PASS: 256 vectors checked, 0 errors (writes mult.vcd)

# 2. Error injection test
sed -i 's/assign sum  = a ^ b ^ cin;/assign sum  = a ^ b;/' rtl/full_adder.v
iverilog -g2012 -o sim_err tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim_err
# -> FAIL: 256 vectors checked, 95 errors

# 3. Restoration
git checkout rtl/full_adder.v
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v && vvp sim
```

```
# -> PASS: 256 vectors checked, 0 errors

# ============================================================================
# Phase 3: Part B — Logic Synthesis & Gate-Level Simulation
# ============================================================================
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    export PDK_ROOT=/foss/pdks; export PDK=sky130A
    yosys -c synth/synth.tcl
"
# -> synth/mult_array_stat.rpt and synth/mult_array_netlist.v (73 cells)

podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    CELLS=/foss/pdks/sky130A/libs.ref/sky130_fd_sc_hd/verilog
    iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
      tb/tb_mult.v \$CELLS/primitives.v \$CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
    vvp gls
"
# -> PASS: 256 vectors checked, 0 errors

# ============================================================================
# Phase 4: Part C — Static Timing Analysis with OpenSTA
# ============================================================================
# Run 1: T = 10.0 ns (100 MHz target)
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    export PDK_ROOT=/foss/pdks; export PDK=sky130A; unset STA_PERIOD
    sta sta/sta.tcl
"
# -> worst slack max 7.40 ns => Tmin = 2.596 ns, Fmax = 385.2 MHz (or 384.6 MHz at 2.60 ns)

# Run 2: T = 2.60 ns confirmation
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    export PDK_ROOT=/foss/pdks; export PDK=sky130A; export STA_PERIOD=2.60
    sta sta/sta.tcl
"
# -> worst slack max 0.00 ns (reported to 2 decimal places, exact slack +0.004 ns)

# ============================================================================
# Phase 5: Part D — Physical Design with LibreLane
# ============================================================================
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array/pnr \
  $IMAGE --skip bash -c "
    source /foss/tools/sak/sak-pdk-script.sh sky130A || true
    librelane --manual-pdk config.json
"
# Final Views Copied:
cp pnr/runs/RUN_2026-10-23_20-08-02/final/metrics.json pnr/metrics.json
cp pnr/runs/RUN_2026-10-23_20-08-02/final/gds/mult_array.gds pnr/mult_array.gds

# KLayout Layout Rendering:
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    source /foss/tools/sak/sak-pdk-script.sh sky130A || true
    QT_QPA_PLATFORM=offscreen klayout -b -r scripts/klayout_shots.py
"
```